

装修费用管家 — 系统设计文档

本文档面向希望理解系统内部实现的开发者/学习者，涵盖设计思路、技术选型、源码脉络和关键机制。

一、项目概览

项目	内容
项目名称	装修费用管家（维佳关山郡）
线上地址	https://decoration-app.pages.dev
代码仓库	https://github.com/robin-yang-hi/decoration-app
部署平台	Cloudflare Pages（免费）
云端存储	Cloudflare KV（免费额度）
定位	个人/家庭装修全周期费用记录 + 预算管理 + 进度跟踪 + AI 分析

二、技术栈与选型理由

2.1 前端核心

技术	版本	选型理由
React	19	生态成熟，Hooks 模型适合数据层抽象
TypeScript	5.9	类型安全，费用/预算等数据结构复杂，TS 能显著减少 bug
Vite	7	构建极快，dev server 秒启动； <code>base: './'</code> 支持子路径部署
Tailwind CSS	v4	原子化 CSS，暖木棕主题通过 CSS 变量统一管理；v4 用 <code>@tailwindcss/vite</code> 插件集成
react-router-dom	v7	用 HashRouter（ <code>#/records</code> ），避免静态托管下刷新 404

2.2 UI 与交互

技术	用途
@radix-ui/*	无样式基础组件（Dialog、Dropdown、AlertDialog、Select、Progress 等），可完全自定义外观
lucide-react	图标库，轻量且与 React 契合
sonner	Toast 通知（导出成功/失败、导入确认等）
react-hook-form + zod	表单校验（费用录入、预算设置）
react-error-boundary	全局错误兜底，崩溃时显示"出错了 + 刷新页面"
vaul	移动端底部抽屉（Sheet）

2.3 数据可视化与导出

技术	用途
echarts + echarts-for-react	分类占比饼图、月度趋势柱状图
html2canvas	将隐藏的 HTML 报告渲染为图片（解决 PDF 中文乱码的关键）
jspdf	将 html2canvas 产出的图片写入 PDF

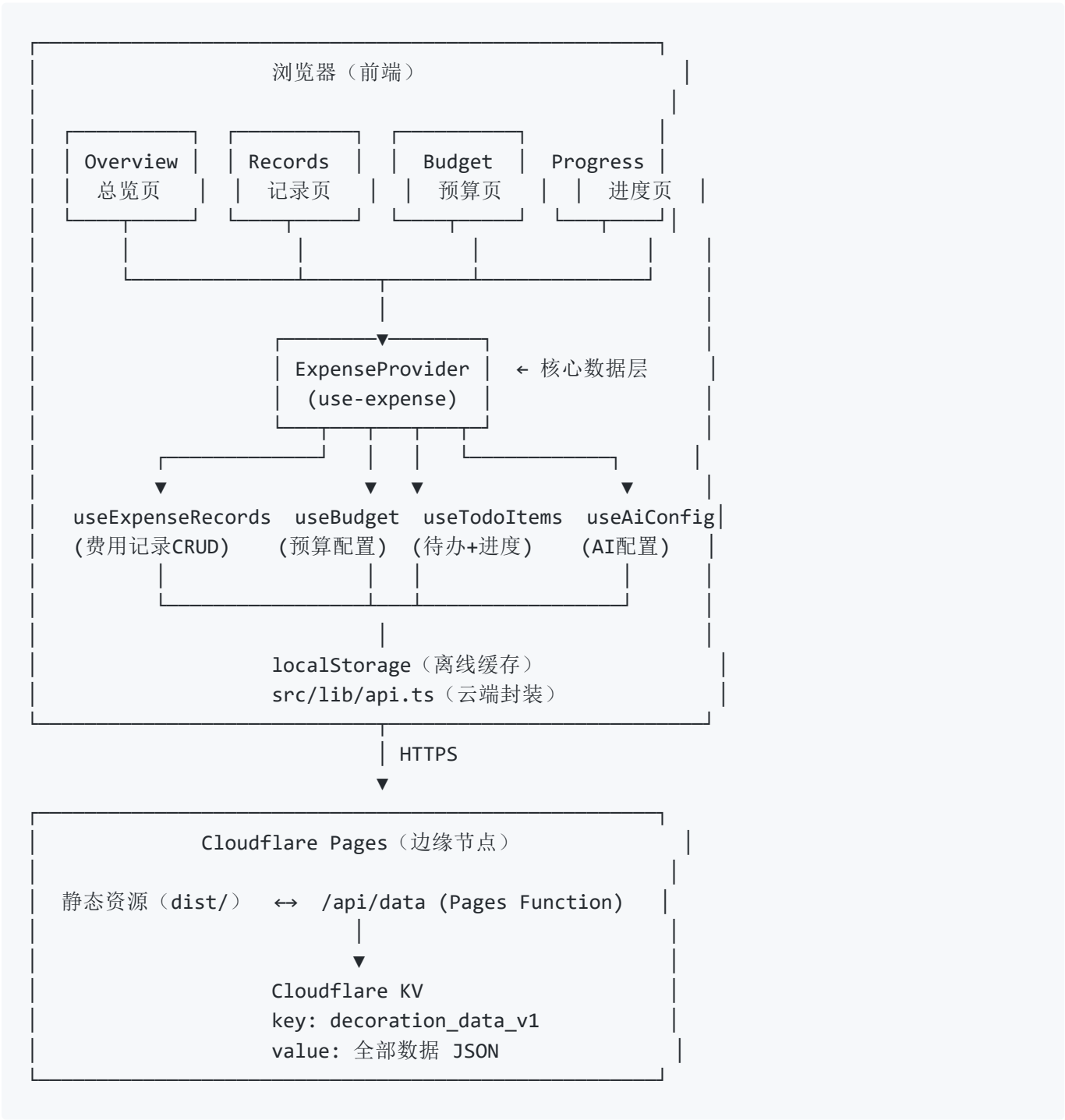
PDF 中文乱码的解决方案：jspdf 直接绘制中文需要嵌入中文字体（文件大、复杂）。本项目改用 html2canvas **先把整份 HTML 报告转成图片，再把图片塞进 PDF**，彻底绕开字体问题，中文渲染完美。

2.4 后端与存储

技术	用途
Cloudflare Pages Functions	无服务器函数，文件放在 <code>functions/api/data.ts</code> ，自动部署为 <code>/api/data</code> 接口
Cloudflare KV	键值存储，免费额度 1GB/天 10 万次读取，完全够用

为什么不用传统服务器？个人项目流量极小，Cloudflare Pages + KV 的免费组合足够，且无需维护服务器、自动 HTTPS、全球 CDN。

三、系统架构



四、源码功能实现脉络

4.1 入口与路由

src/index.tsx

- HashRouter：URL 用 #/ 前缀，静态托管刷新不 404
- ErrorBoundary：全局错误兜底，崩溃时显示友好页面 + 刷新按钮
- 引入 index.css (Tailwind 入口 + 自定义主题变量)

src/app.tsx

- 4 个路由全部包裹在 <Layout> 下：
 - / → OverviewPage (总览)
 - /records → RecordsPage (费用记录)
 - /budget → BudgetPage (预算设置)
 - /progress → ProgressPage (装修进度)

4.2 布局与导航

src/components/Layout.tsx

- 用 <ExpenseProvider> 包裹整个应用，所有页面共享同一份数据
- 桌面端：顶部 Header 导航
- 移动端：底部固定导航栏 (4 个 tab)，适配安全区 env(safe-area-inset-bottom)
- <Toaster /> 全局 toast

src/components/Header.tsx

- 左侧：Logo + 标题"装修费用管家" + 小号后缀"维佳关山郡"
- 中间：4 个导航链接 (桌面端)
- 右侧：4 个导出/导入按钮 (PDF / CSV / 备份 / 导入)
- 移动端：汉堡菜单 → Sheet 抽屉，包含导航 + 导出功能
- 导入时弹出 AlertDialog 二次确认 (防止误覆盖)

4.3 核心数据层 (最重要)

src/hooks/use-expense.tsx — ExpenseProvider

这是整个系统的大脑，做了三件事：

① 聚合 4 个子 hook 的数据和操作

```
useExpenseRecords → records, addRecord, updateRecord, deleteRecord
useBudgetConfig   → budget, saveBudget
useTodoItems      → categories, updateItemStatus, overallProgress, categoryProgress
useAiConfig       → config, saveAiConfig
```

② 云端同步机制（核心设计）

```
启动时
├── fetchCloudData() → GET /api/data
│   ├── 云端有实际数据（records非空 / budget非空 / todoStatus有key / aiConfig有
│   │   └── 用云端数据覆盖各 hook 的本地状态
│   └── 云端为空或拉取失败
│       ├── 保留本地 localStorage 数据
│       └── 主动上传一次本地数据到云端（初始化）
└── hasSyncedRef = true（标记首次同步完成）

运行中
└── useEffect 监听 [records, budget, todoCategories, aiConfig]
    └── 数据变化 → 防抖 1.5 秒 → saveCloudData() → POST /api/data

手动同步
└── syncNow() → 立即上传 + 重新从云端拉取（解决多端冲突）
```

关键设计决策：

- **防抖 1.5 秒**：用户连续输入时不频繁发请求，停止操作 1.5 秒后才保存
- **hasSyncedRef**：首次云端拉取期间不触发保存，避免空数据覆盖云端
- **空对象不算有数据**：`todoStatus: {}` 是 falsy 判断，必须 `Object.keys().length > 0` 才算有数据（曾因这个 bug 导致本地被空数据覆盖）
- **localStorage 保留**：各子 hook 仍写 localStorage，作为离线缓存和降级方案

③ 导出/导入

- `exportAllData()`：聚合全部数据为 JSON（records + budget + todoStatus + aiConfig）
- `importAllData(jsonStr)`：解析 JSON 后分别恢复到各子 hook

4.4 子 hook 设计

每个子 hook 遵循相同模式：

```
useState(初始值)
  ↓
useEffect(() => { 从 localStorage 读取 → setState })
  ↓
useCallback 操作函数（修改 state + 写 localStorage）
  ↓
return { state, 操作函数, setXxx（供云端覆盖用） }
```

Hook	localStorage Key	核心能力
useExpenseRecords	__app_decoration_expense_records	增删改查费用记录，id 用 <code>rec_\${Date.now()}_\${random}</code>
useBudgetConfig	__app_decoration_budget_config	总预算 + 7 个分类预算，读取时合并默认值
useTodoItems	__app_decoration_todo_items	只存 status map（不存整个分类结构），启动时从默认配置重建 + 恢复状态；计算总体/分类完成度
useAiConfig	__app_decoration_ai_config	apiKey + model + apiBase，默认指向豆包方舟平台

待办状态的存储技巧：不存整个 76 项的数组，只存 `{itemId: status}` 的 map。因为分类结构是写死在代码里的（`TODO_CATEGORIES`），启动时用默认结构 + status map 重建，既省空间又方便升级分类结构。

4.5 云端 API

src/lib/api.ts — 前端封装

- `fetchCloudData()`：GET `/api/data`，失败返回 null（降级）
- `saveCloudData(data)`：POST `/api/data`，返回 boolean
- `CloudData` 接口： `{ version, updatedAt, records, budget, todoStatus, aiConfig }`

functions/api/data.ts — Cloudflare Pages Function

- `onRequestGet`：从 KV 读取 `decoration_data_v1`，返回 JSON 或 `{}`

- `onRequestPost` : 校验 JSON 后写入 KV, 返回 `{ok: true, savedAt}`
- `onRequestOptions` : CORS 预检返回 204
- KV 绑定变量名: `DECO_DATA` (在 Cloudflare Dashboard → Pages → Settings → Functions → KV bindings 配置)
- 所有响应带 `Cache-Control: no-store`, 确保不被 CDN 缓存

4.6 数据配置层 (`src/data/`)

文件	内容
<code>input-number.ts</code>	<code>IExpenseRecord</code> 类型、7 个费用分类联合类型、支付方式类型、3 条示例数据
<code>category-config.ts</code>	分类中文标签、emoji 图标、子分类建议 (录入时的下拉选项)、支付方式标签、图表配色
<code>budget.ts</code>	<code>IBudgetConfig</code> 类型 (<code>totalBudget + categoryBudgets</code>)、默认零预算
<code>todo-config.ts</code>	10 大分类 76 项待办清单、4 种状态 (<code>done/in_progress/pending/not_applicable</code>)、状态中文标签

4.7 页面实现

OverviewPage (总览页)

- 4 个统计卡片: 总支出、预算使用率、记录笔数、装修进度
- 装修进度总览: 总体进度条 + 10 个分类进度条
- 分类费用占比: echarts 饼图
- 月度支出趋势: echarts 柱状图 (按月聚合)
- AI 分析按钮 → `AiAnalysisDialog`
- 云端同步状态指示器 ( 已同步 /  同步中 /  离线) + 立即同步按钮

RecordsPage (费用记录页)

- 按月份分组展示, 月份间有明显间隔区分
- 每条记录可编辑/删除
- 新增记录表单 (react-hook-form + zod 校验)
- 分类筛选

BudgetPage (预算设置页)

- 总预算输入

- 7 个分类预算输入
- 实时显示已花费/预算/使用率/剩余

ProgressPage (装修进度页)

- 10 大分类手风琴展开
- 每项 4 个状态按钮 (已完成/进行中/待启动/暂不涉及)
- 每类完成度百分比
- 一键重置

4.8 PDF 导出 (`src/lib/pdf-export.ts`)

构建隐藏 HTML 报告 (width: 794px = A4宽度)

- 包含: 总览卡片、分类明细表、月度记录表、待办进度
- `html2canvas` 渲染为 `canvas`
- 按 A4 尺寸切片
- `jspdf.addImage()` 逐页写入
- 保存为 PDF

关键: HTML 里用 `font-family: 'Noto Sans SC'`, `html2canvas` 能正确渲染中文, 图片进 PDF 后不存在字体问题。

4.9 CSV 导出

- 带 BOM (`\uFEFF`), Excel 打开不乱码
- 字段: 日期、类别、子类别、金额、支付方式、商家、备注
- 含逗号/引号/换行的字段用双引号包裹并转义

五、部署架构

git push → GitHub → Cloudflare Pages 自动构建

```
|
├─ Build command: npm run build
├─ Output directory: dist
└─ Functions 目录自动部署为 API
```



`https://decoration-app.pages.dev`

静态资源走 CDN, `/api/*` 走边缘函数

本地开发：

```
npm install
npm run dev      # http://localhost:5173
npm run build    # tsc 类型检查 + vite 构建
npm run preview  # 预览构建产物
```

六、关键设计决策总结

决策	理由
HashRouter 而非 BrowserRouter	静态托管刷新不 404，无需服务端 rewrite
KV 存全量 JSON 而非分 key	简单可靠，个人项目数据量小（几 KB），全量读写无性能问题
防抖 1.5 秒保存	平衡实时性和请求频率
localStorage 双写	离线可用 + 云端故障时降级
html2canvas + jsPDF	彻底解决 PDF 中文乱码
待办只存 status map	分类结构写死在代码里，存储最小化，方便后续升级分类
空对象不算"有数据"	防止 {} 误判导致本地被覆盖
暖木棕配色	装修场景温馨感，CSS 变量统一管理

七、已知限制与可改进方向

- 多端非实时同步：**当前是拉取模式（刷新页面才获取最新），如需实时可加 30 秒轮询或 WebSocket
- 无用户认证：**KV 是单用户共享，任何人知道网址都能修改数据。如需多用户需加认证
- AI 分析为本地规则：**AiAnalysisDialog 当前是本地计算的模板化建议，接入真实豆包 API 需用户配置 apiKey
- 无数据版本历史：**KV 每次覆盖写，误删无法回滚（建议定期用"备份"按钮导出 JSON）
- KV 最终一致性：**Cloudflare KV 是最终一致性，极端情况下两端同时修改可能有几秒延迟